



# UNIVERSIDAD AUTONOMA DE SINALOA



## SOFTWARE DE SISTEMAS

GRUPO 4-01 INGENIERIA DE SOFTWARE

### EJERCICIOS PROCESO DEL ANÁLISIS LÉXICO

**ALUMNO:**

ESCALANTE SIANUQUI IRVING JARED

**FECHA:**

10/10/2025

### 1. ¿Cuál es la función principal del analizador léxico dentro de un compilador?

El **analizador léxico** (o *lexer*) tiene como función principal **leer el código fuente y convertirlo en una secuencia de tokens**. Básicamente, toma los caracteres del programa y los agrupa en unidades significativas que el compilador pueda entender, descartando espacios en blanco y comentarios.

### 2. ¿Qué es un token y en qué se diferencia de un lexema?

- **Token:** Es una categoría general de elementos del lenguaje, como *identificador*, *número*, *operador*, etc. Representa **la clase a la que pertenece un lexema**.
- **Lexema:** Es la **cadena concreta de caracteres** en el código que coincide con un token.

#### **Ejemplo:**

Código: contador

- Token → IDENTIFICADOR
- Lexema → "contador"

### 3. Menciona al menos tres tipos comunes de tokens en un lenguaje de programación.

1. **Identificadores:** nombres de variables, funciones, etc.
2. **Literales:** números, cadenas de texto, caracteres.
3. **Operadores:** +, -, \*, /, =, etc.  
Otros posibles: palabras reservadas, símbolos de puntuación (;, ,, (, )).

### 4. Explica qué papel cumple la tabla de símbolos durante el análisis léxico.

La **tabla de símbolos** es una estructura de datos que **almacena información sobre los identificadores encontrados**, como su nombre, tipo de dato, dirección en memoria o ámbito. Sirve para:

- Evitar duplicados.
- Facilitar la verificación de tipos y el enlace de variables en fases posteriores del compilador.

**5. ¿Por qué se dice que el análisis léxico es la primera fase del proceso de compilación?**

Porque **es el primer paso en el que el compilador analiza el código fuente**, transformando una cadena de caracteres en tokens que puedan ser procesados por las fases siguientes: análisis sintáctico, semántico, optimización y generación de código.

**6. Dado el siguiente fragmento de código en C:**

total = suma + 25 \* contador;

**a) Divide la línea en sus tokens:**

1. total
2. =
3. suma
4. +
5. 25
6. \*
7. contador
8. ;

**b) Clasifica cada token:**

| Token    | Tipo                |
|----------|---------------------|
| Total    | Identificador       |
| =        | Operador            |
| Suma     | Identificador       |
| +        | Operador aritmético |
| 25       | Numero              |
| *        | Operador aritmético |
| Contador | Identificador       |
| ;        | Símbolo             |

**7. Diseña una lista de reglas léxicas simples (pseudocódigo o regex informal):**

- **Números enteros:**

entero ::= [0-9] +

- **Identificadores que comiencen con una letra:**

identificador ::= [a-zA-Z][a-zA-Z0-9\_]\*

- **Símbolo de asignación (=):**

asignacion ::= "="

- ***\*Operadores aritméticos (+, -, \*, /):***

operador\_aritmetico ::= "+" | "-" | "\*" | "/"